Theory of Programming Languages

2. Lambda Calculus

Kihong Heo

KAIST

Lambda Calculus람다계산법

- Introduced by Alonzo Church (1935)
- Answer to the grand question: “What does computable function mean?”
  - Recursive function (Gödel), Turing machine (Turing), Lambda calculus (Church)
  - All three definitions are equivalent
  - “Any computable functions are simulated by a Turing machine” (Church-Turing Thesis)
- Foundation of programming languages
- Syntax (문법구조): $E \quad \rightarrow \quad x \mid \lambda x.E \mid E\ E$

Scope유효범위

- x is said to be **bound** when it occurs in the body E of an abstraction $\lambda x\,.\,E$
- x is said to be **free** if it is not bound
- Example
  - $\lambda x\,.\,x\ y$
  - $\lambda x\,.\,\lambda y\,.\,y\ (\lambda x\,.\,x)\ x$

Semantics의미구조

- Semantics: concern with the **meaning** of grammatically correct programs
  - interpreters execute programs w.r.t. the **semantics**
  - compilers translate programs w.r.t. the **semantics**
  - analyzers analyze programs w.r.t. the **semantics**

```
x := 1; y := x + 1;
```
equivalent? / why?
```
a := 1; b := a + 1;
```

```
x := 1; y := x; print(x);
```
correct? / why?
```
x := 1; print(x);
```

Different Styles of Semantics

- **Operational Semantics (과정형/어떻게 의미구조)**: "**How** to compute the execution result"
  - so-called transitional style (상태전이식)
  - `3 * (2 + 1)` : **3 * (2 + 1) → 3 * 3 → 9**
- **Denotational Semantics (직접형/무엇 의미구조)**: "**What** the execution result is"
  - so-called compositional style (조립식)
  - `3 * (2 + 1)` **: 9**
- **…**

Different approaches for different purposes and languages

Operational Semantics과정형 의미구조

- Program semantics is a series of machine state transitions
  - Big-step (큰걸음): how the **overall results** of executions are obtained
  - Small-step (잰걸음): how the **individual steps** of the computations take place
- In this lecture, we will mainly use small-step operational semantics

Big-step

$$\dfrac{\dfrac{\overline{2 \times 2 \Rightarrow 4} \quad \overline{4 \times 2 \Rightarrow 8}}{2 \times 2 \times 2 \Rightarrow 8} \quad \dfrac{\overline{2 + 1 \Rightarrow 3} \quad \overline{8 \times 3 \Rightarrow 24}}{}}{(2 \times 2 \times 2) \times (2 + 1) \Rightarrow 24}$$

why?

Small-step

$$\begin{aligned} & (2 \times 2 \times 2) \times (2 + 1) \\ \rightarrow\ & (4 \times 2) \times (2 + 1) \\ \rightarrow\ & 8 \times (2 + 1) \\ \rightarrow\ & 8 \times 3 \\ \rightarrow\ & 24 \end{aligned}$$

so?

Small-Step Semantics잰걸음 의미구조

- Redex (reducible expression): $(\lambda x.E_1)\ E_2$
- β-reduction: $(\lambda x.E_1)\ E_2 \rightarrow E_1\{E_2/x\}$
- Different reduction strategies: full β-reduction, normal order, call-by-value (CBV), call-by-name (CBN), etc
- A term is in normal form if it cannot be reduced by β-reduction

$$(\lambda x.x)(\lambda y.y\ y)(\lambda z.z) \rightarrow (\lambda y.y\ y)(\lambda z.z) \rightarrow (\lambda z.z)(\lambda z.z) \rightarrow (\lambda z.z)$$

Example: Full β-Reduction

- Any redex can be reduced at any time

$$\frac{}{(\lambda x.E_1)\ E_2 \to E_1\{E_2/x\}} \qquad \frac{E_1 \to E_1'}{E_1\ E_2 \to E_1'\ E_2} \qquad \frac{E_2 \to E_2'}{E_1\ E_2 \to E_1\ E_2'} \qquad \frac{E \to E'}{\lambda x.E \to \lambda x.E'}$$

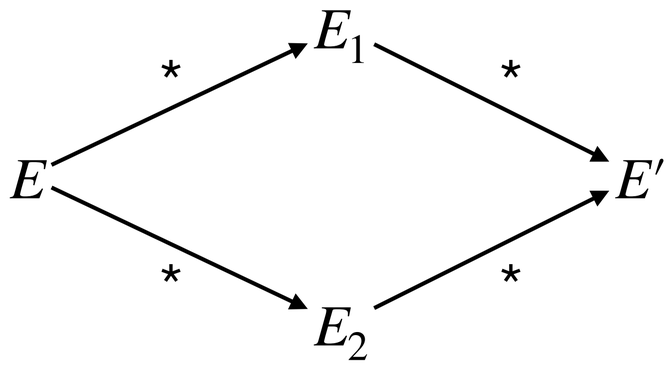
- Example

$$(\lambda x.x)((\lambda y.y)(\lambda z.(\lambda w.w)z))$$

Confluence만남

Theorem (Church-Rosser). Let E be an expression. For some expressions E_1 and E_2 , if $E \to^* E_1$ and $E \to^* E_2$ then

$$\exists E' \,.\, E_1 \to^* E' \wedge E_2 \to^* E'$$



Example: Call-by-name (CBN)

- Reduce the leftmost outermost redex
  - Function application even when its argument is not a value (lazy evaluation)
  - Used in languages such as Haskell, OCaml (with “lazy”), etc

$$\frac{}{(\lambda x.E_1)\ E_2 \to E_1\{E_2/x\}} \qquad \frac{E_1 \to E_1'}{E_1\ E_2 \to E_1'\ E_2}$$

- Example

$(\lambda x.x)((\lambda y.y)(\lambda z.(\lambda w.w)z))$

$(\lambda x.\lambda y.x)((\lambda z.z)(\lambda w.w))$

$((\lambda x.x)(\lambda y.y))((\lambda z.z)((\lambda w.w)(\lambda v.v)))$

Example: Call-by-value (CBV)

- Reduce the leftmost outermost redex when its right-hand side is a value ($\lambda x.E$)
  - Arguments evaluated fully before function applications (eager evaluation)
  - Widely used in mainstream languages such as OCaml, C/C++, Java, etc

$$\frac{}{(\lambda x.E)\ v \to E\{v/x\}} \qquad \frac{E_1 \to E_1'}{E_1\ E_2 \to E_1'\ E_2} \qquad \frac{E_2 \to E_2'}{v_1\ E_2 \to v_1\ E_2'}$$

- Example

  $(\lambda x.x)((\lambda y.y)(\lambda z.(\lambda w.w)z))$

  $(\lambda x.\lambda y.x)((\lambda z.z)(\lambda w.w))$

Cf. Big-Step Semantics큰걸음 의미구조

- $E \Downarrow v$: Term E evaluates to value v
- Rule (CBV):

$$\frac{}{\lambda x.E \Downarrow \lambda x.E} \qquad \frac{E_1 \Downarrow \lambda x.E_3 \qquad E_2 \Downarrow v \qquad E_3\{v/x\} \Downarrow v'}{E_1\ E_2 \Downarrow v'}$$

Theorem (Equivalence). For every term E and value v,

$$E \Downarrow v \iff E \to^* v$$

Substitution

- The main part of the β-reduction: $E_1\{E_2/x\}$
- Example: $(\lambda x.x\ x)(\lambda y.y) \rightarrow (x\ x)\{\lambda y.y/x\} = (\lambda y.y)(\lambda y.y)$
- Looks simple but actually quite tricky

Substitution: First Attemp

- Naive definition

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$
$$(\lambda y.E_1)\{E/x\} = \lambda y.E_1\{E/x\}$$
$$(E_1\ E_2)\{E/x\} = (E_1\{E/x\})\ (E_2\{E/x\})$$

- Problem?
- Example: $(\lambda x.x)\{y/x\}$
- Why? Incorrectly change bound variables
- How to fix? Don't substitute bound variables

Substitution: Second Attemp

- Idea: stop substitution for bound variables

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \begin{cases} \lambda y.E_1 & \text{if } y = x \\ \lambda y.E_1\{E/x\} & \text{otherwise} \end{cases}$$

$$(E_1\ E_2)\{E/x\} = (E_1\{E/x\})\ (E_2\{E/x\})$$

- Problem?
- Example: $(\lambda z.x)\{z/x\}$
- Why? Free variables can become bound variables (so-called variable capture)
- How to fix? Don't substitute if the param = a free variable of the target expression

Free Variable

- Variables not bound in an abstraction

$$
\begin{aligned}
FV(x) &= \{x\} \\
FV(E_1\ E_2) &= FV(E_1) \cup FV(E_2) \\
FV(\lambda x.E) &= FV(E) \setminus \{x\}
\end{aligned}
$$

Substitution: Third Attemp

- Avoid variable capture

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \begin{cases} \lambda y.E_1 & \text{if } y = x \\ \lambda y.E_1\{E/x\} & \text{if } y \neq x \wedge y \notin FV(E) \end{cases}$$

$$(E_1\ E_2)\{E/x\} = (E_1\{E/x\})\ (E_2\{E/x\})$$

- Problem?
- Example: $(\lambda y.x\ y)\{(y\ z)/x\}$
- Why? no applicable rule
- But, the name of the bound variable y really matter?

α-Conversion

- Terms that differ only in the bound variable names are interchangeable in all contexts
- Two terms are equivalent up to renaming of bound variables

$$\lambda x.x\ y = \lambda z.z\ y$$

Substitution: Final Version

- Introduce a fresh variable z when necessary

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \begin{cases} \lambda y.E_1 & \text{if } y = x \\ \lambda y.E_1\{E/x\} & \text{if } y \neq x \wedge y \notin FV(E) \\ (\lambda z.E_1\{y/z\})\{E/x\} & \text{if } y \neq x \wedge y \in FV(E) \end{cases}$$

$$(E_1\ E_2)\{E/x\} = (E_1\{E/x\})\ (E_2\{E/x\})$$

- Conventionally, implicitly assume α-conversion

$$y\{E/x\} = \begin{cases} E & \text{if } y = x \\ y & \text{otherwise} \end{cases}$$

$$(\lambda y.E_1)\{E/x\} = \lambda y.E_1\{E/x\} \qquad \text{if } y \neq x \wedge y \notin FV(E)$$

$$(E_1\ E_2)\{E/x\} = (E_1\{E/x\})\ (E_2\{E/x\})$$

Evaluation Context Semantics실행문맥 의미구조

- A convenient way to define operational semantics
  - Separate rules into computation (계산) and congruence (나란한) rules
- Operational semantic rule = program rewriting rule
- Where to rewrite? By the evaluation contexts
- How to rewrite? By the rewriting rules

$$\frac{}{(\lambda x.E)\ v \to E\{v/x\}}$$

$$\frac{E_1 \to E_1'}{E_1\ E_2 \to E_1'\ E_2}$$

$$\frac{E_2 \to E_2'}{v_1\ E_2 \to v_1\ E_2'}$$

- Example $((\lambda x.x)(\lambda y.y))((\lambda z.z)(\lambda w.w)) \to (\lambda y.y)((\lambda z.z)(\lambda w.w)) \to (\lambda y.y)(\lambda w.w) \to \lambda w.w$

Evaluation Context실행문맥

- Evaluation context K : program with rewrite location
- Term with [ ] (hole), which will be rewritten with a new expression
- K is defined using a grammar
- $K[\ ]$ (or simply K) : program with rewriting target
- $K[E]$: program where the hole has been filled with E
- Example: $((\lambda x.x)(\lambda y.y))((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)((\lambda z.z)(\lambda w.w)) \rightarrow (\lambda y.y)(\lambda w.w) \rightarrow \lambda w.w$

Evaluation Context for Lambda Calculus

- Evaluation contexts for CBV

$$\begin{array}{rcl} E & \rightarrow & x \mid \lambda x.E \mid E\ E \\ v & \rightarrow & \lambda x.E \\ K & \rightarrow & [\ ] \mid K\ E \mid v\ K \end{array}$$

$$\frac{}{(\lambda x.E)\ v \rightarrow E\{v/x\}}$$

$$\frac{E \rightarrow E'}{K[E] \rightarrow K[E']}$$

- Example

$$\begin{array}{l} K_1 = [\ ](\lambda x.x) \\ K_2 = (\lambda z.z\ z)[\ ] \\ K_3 = ([\ ]\lambda x.x\ x)((\lambda y.y)(\lambda y.y)) \\ K_4 = [\ ]((\lambda z.z)(\lambda w.w)) \end{array}$$

$$\begin{array}{rl} K_1[\lambda y.y\ y] = & (\lambda y.y\ y)\lambda x.x \\ K_2[\lambda x.\lambda y.x] = & (\lambda z.zz)(\lambda x.\lambda y.x) \\ K_3[\lambda f.\lambda g.f\ g] = & ((\lambda f.\lambda g.f\ g)\lambda x.x\ x)((\lambda y.y)(\lambda y.y)) \\ K_4[(\lambda x.x)(\lambda y.y)] = & (\lambda x.x)(\lambda y.y)((\lambda z.z)(\lambda w.w)) \\ K_4[\lambda y.y] = & (\lambda y.y)((\lambda z.z)(\lambda w.w)) \end{array}$$

Programming in the Lambda Calculus

- “Any computable functions are simulated by a Turing machine” (Church-Turing Thesis)
  - Equivalently, by the lambda calculus
- Concepts in high-level PL can be represented in the lambda calculus
  - E.g., Boolean, natural number, conditional expression, loop, recursion, etc
- Idea: value producer (introduction) / consumer (elimination)

Church Booleans

$$E \to \ldots \mid \texttt{true} \mid \texttt{false} \mid \texttt{if}\ E\ E\ E \mid E \wedge E \mid E \vee E \mid \neg E$$

- Value producer: constants true and false
- Value consumer: test, negation, conjunction, disjunction, etc
- Intuition: boolean value = selector

$$\begin{aligned}
\texttt{true} &= \lambda t.\lambda f.t \\
\texttt{false} &= \lambda t.\lambda f.f \\
\texttt{if} &= \lambda l.\lambda m.\lambda n.l\ m\ n \\
\texttt{and} &= \lambda b.\lambda c.b\ c\ \texttt{false} \\
\texttt{or} &= \lambda b.\lambda c.b\ \texttt{true}\ c \\
\texttt{not} &= \lambda b.\texttt{false}\ \texttt{true}
\end{aligned}$$

Example

- `if true` v w
- `and true true`
- `and true false`

Pairs

$$E \to \ldots \mid \langle E, E \rangle \mid \texttt{fst}\ E \mid \texttt{snd}\ E$$

- Value producer: pairing of two values
- Value consumer: projection
- Intuition: pair = storage parameterized by selector

$$\begin{aligned} \texttt{pair} &= \lambda f.\lambda s.\lambda b.b\ f\ s \\ \texttt{fst} &= \lambda p.p\ \texttt{true} \\ \texttt{snd} &= \lambda p.p\ \texttt{false} \end{aligned}$$

Example

- $\texttt{fst}(\texttt{pair}\ v\ w)$

Church Numerals

$$E \to \texttt{0} \mid \texttt{S}\ E \mid E + E \mid E \times E \mid E^E$$

- Value producer: natural numbers
- Value consumer: successor, plus, times, etc
- Intuition: natural number = # function applications

$$\begin{aligned}
\texttt{zero} &= \lambda s.\lambda z.z \\
\texttt{one} &= \lambda s.\lambda z.s\ z \\
\texttt{two} &= \lambda s.\lambda z.s\ (s\ z) \\
\texttt{succ} &= \lambda n.\lambda s.\lambda z.s\ (n\ s\ z) \\
\texttt{plus} &= \lambda m.\lambda n.\lambda s.\lambda z.m\ s\ (n\ s\ z)
\end{aligned}$$

Loop

- Divergent combinator $\Omega = (\lambda x.x\ x)(\lambda x.x\ x)$
  - Cannot be evaluated to a normal form
- Fixpoint combinator

$\texttt{fix} = \lambda f.(\lambda x.f\ (\lambda y.x\ x\ y))\ (\lambda x.f\ (\lambda y.x\ x\ y))$

Recursion

- How can we define a recursive function using the lambda calculus?

```
let rec factorial n =
  if n = 0 then 1
  else n * factorial (n - 1)
```

$$\texttt{factorial} = \lambda n. \begin{cases} 1 & \text{if } n = 0 \\ n \times \texttt{factorial}(n-1) & \text{otherwise} \end{cases}$$

- Idea: fixpoint combinator

Definition as Fixpoint (1)

- Consider this inductive definition as an equation

$$\texttt{factorial} = \lambda n. \begin{cases} 1 & \text{if } n = 0 \\ n \times \texttt{factorial}(n-1) & \text{otherwise} \end{cases}$$

$$\texttt{factorial} = \mathcal{F}(\texttt{factorial})$$

$$\text{where } \mathcal{F}(X) = \lambda n. \begin{cases} 1 & \text{if } n = 0 \\ n \times X(n-1) & \text{otherwise} \end{cases}$$

Definition as Fixpoint (2)

- What is the definition of factorial? **A solution to this equation!**

$$\texttt{factorial} = \mathcal{F}(\texttt{factorial}) \quad \Longleftrightarrow \quad X = \mathcal{F}(X) \text{ where } X = \texttt{factorial}$$

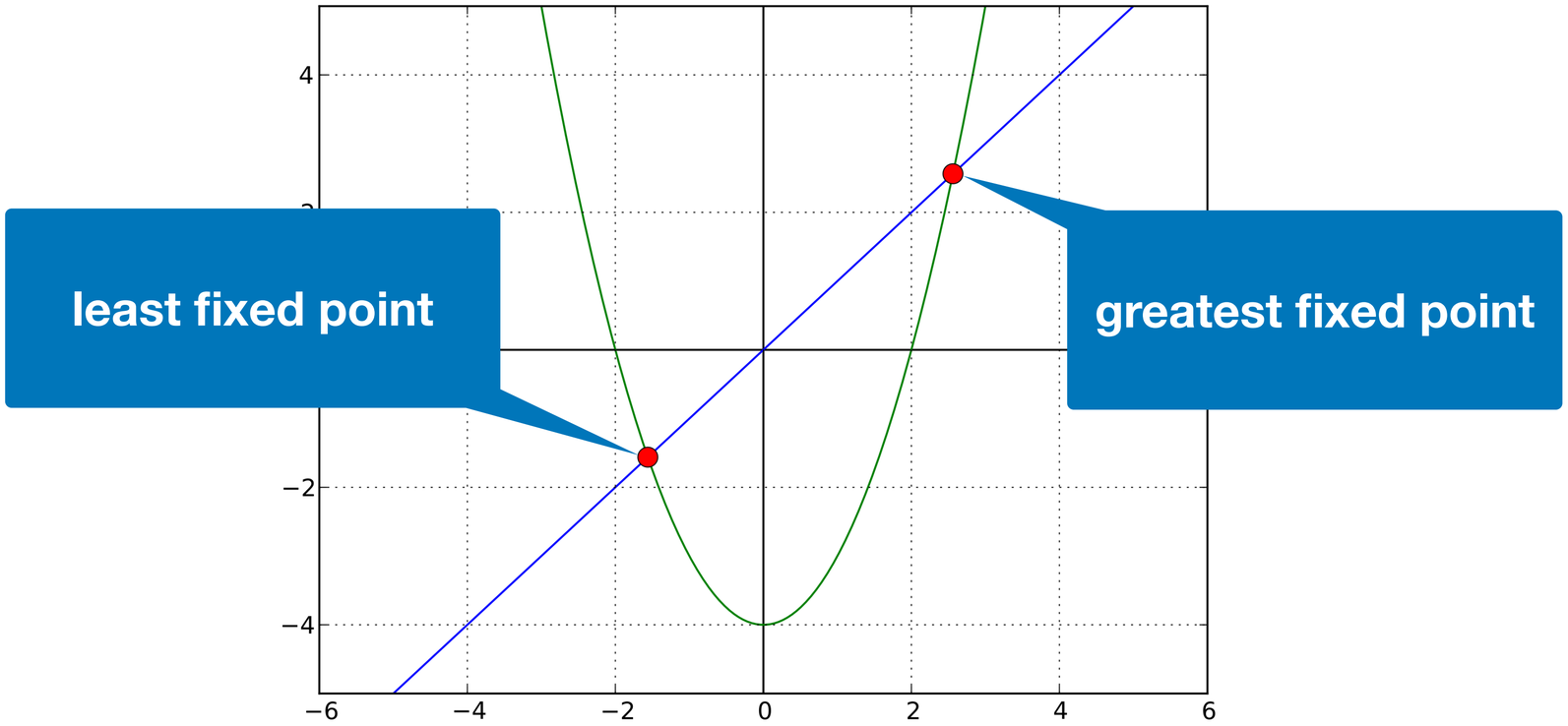
- Solution: a fixed point of $\mathcal{F}$

$$\texttt{factorial} = \texttt{fix}\ \mathcal{F}$$

$$\mathcal{F} = \lambda X.\lambda n.\texttt{if}\ \text{eq}\ n\ 0\ \texttt{then}\ 1\ \texttt{else}\ \text{times}\ n\ (X\ (\text{pred}\ n)))$$

Exercise: Fixed Point

$$f(x) = x^2 - 4$$



*https://en.wikipedia.org/wiki/Least_fixed_point

Exercise: Fixed Point

$$\mathbb{N} = \{0\} \cup \{n + 1 \mid n \in \mathbb{N}\}$$

$\mathbb{N}$ is the least fixed point of F where $F(X) = \{0\} \cup \{n + 1 \mid n \in X\}$

$$\therefore \mathbb{N} = \mathit{fix}\big(\lambda X.\{0\} \cup \{n + 1 \mid n \in X\}\big)$$

Exercise: Fixed Point

$$\mathbb{L} = \{\mathsf{nil}\} \cup \{0 :: l \mid l \in \mathbb{L}\}$$

$\mathbb{L}$ is the least fixed point of F where $F(X) = \{\mathsf{nil}\} \cup \{0 :: l \mid l \in X\}$

$$\therefore \mathbb{L} = \mathit{fix}\left(\lambda X.\{\mathsf{nil}\} \cup \{0 :: l \mid l \in X\}\right)$$

Exercise: Fixed Point

$$\texttt{reach}(N) = N \ \cup \ \texttt{reach}(\texttt{next}(N))$$

$$\texttt{reach} = \lambda N.N \ \cup \ \texttt{reach}(\texttt{next}(N))$$

$\texttt{reach}$ is the least fixed point of F where $F(X) = \lambda N.N \cup X(\texttt{next}(N))$

$$\therefore \texttt{reach} = \mathit{fix}\left(\lambda X.(\lambda N.N \cup X(\texttt{next}(N))\right)$$

Exercise: Fixed Point

$$\texttt{fact}(N) = \texttt{if } N = 0 \texttt{ ? } 1 \texttt{ : } N * \texttt{fact}(N-1)$$

$$\texttt{fact} = \lambda N.\texttt{if } N = 0 \texttt{ ? } 1 \texttt{ : } N * \texttt{fact}(N-1)$$

$\texttt{fact}$ is the least fixed point of F where $F(X) = \lambda N.\texttt{if } N = 0 \texttt{ ? } 1 \texttt{ : } N * X(N-1)$

$$\therefore \texttt{fact} = \mathit{fix}\big(\lambda X.(\lambda N.\texttt{if } N = 0 \texttt{ ? } 1 \texttt{ : } N * X(N-1))\big)$$

Summary

- Lambda calculus: foundation of programming languages
- Simple (variable, abstraction, application) but general enough (e.g., Church encoding)
  - Church-Turing thesis
- Various evaluation strategies
  - CBV: OCaml, C/C++, Java, etc
  - CBN: Haskell, etc